8 · Exp 9 (probable) — Stack using an array

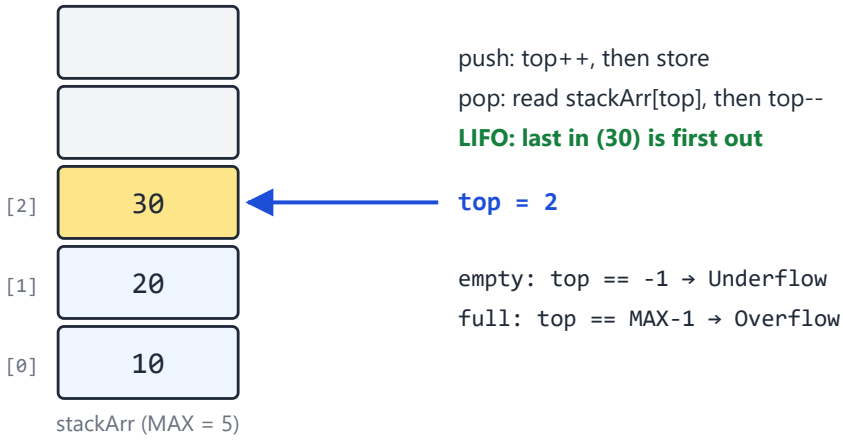
Not in your uploaded files. No Exp 9 file arrived, but Exp 10's figures are captioned "Enqueue Operation on Stack", which points to a stack experiment just before it. This chapter is written in exactly the style of Exp 10, so if Exp 9 is Stack, you're covered. Confirm with a classmate.

One open end (top) → push: top++, store → pop: read, top-- → LIFO

8.1 The idea

A **stack** is a pile: you add and remove only at the **top**. The last item pushed is the first popped: **LIFO** (Last In, First Out). Examples: undo history, the browser back button, function calls (the "call stack" quick sort's recursion uses).

> **GUESS FIRST**
> Push 10, 20, 30, then pop twice. Which values come out, in what order, and what is left?



An array stack after pushing 10, 20, 30. top is the index of the top element; −1 means empty.

8.2 The program

New C++ here (also used in Exp 10):

- `#define MAX 5`: the compiler replaces every `MAX` with 5. Change it in one place to resize.
- **Global variables** (declared above all functions): `stackArr` and `top` are shared by `push`, `pop`, `peek`, `display` and `main`.
- **Menu loop:** `do { show menu; read choice; act } while (choice != 5);` runs until the user picks Exit.

E09_STACK.CPP

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;                                   // -1 = empty
```

```
void push(int value)
{
    if (top == MAX - 1)
        cout << "Stack Overflow!" << endl;
    else
    {
        top++;
        stackArr[top] = value;
        cout << value << " pushed into stack." << endl;
    }
}

void pop()
{
    if (top == -1)
        cout << "Stack Underflow!" << endl;
    else
    {
        cout << stackArr[top] << " popped from stack." << endl;
        top--;
    }
}

void peek()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
        cout << "Top element is " << stackArr[top] << endl;
}

void display()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
    {
        cout << "Stack elements (top to bottom):" << endl;
        for (int i = top; i >= 0; i--)
            cout << stackArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Peek" << endl;
        cout << "4. Display" << endl;
        cout << "5. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to push: ";
```

```
            cin >> value;
            push(value);
        }
        else if (choice == 2)
            pop();
        else if (choice == 3)
            peek();
        else if (choice == 4)
            display();
        else if (choice == 5)
            cout << "Program terminated." << endl;
        else
            cout << "Invalid choice!" << endl;
    } while (choice != 5);

    return 0;
}
```

Push 10, 20, 30 → display → peek → pop all three → pop again (underflow) → exit:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed into stack.
Enter your choice: 1
Enter value to push: 20
20 pushed into stack.
Enter your choice: 1
Enter value to push: 30
30 pushed into stack.
Enter your choice: 4
Stack elements (top to bottom):
30 20 10
Enter your choice: 3
Top element is 30
Enter your choice: 2
30 popped from stack.
Enter your choice: 4
Stack elements (top to bottom):
20 10
Enter your choice: 2
20 popped from stack.
Enter your choice: 2
10 popped from stack.
Enter your choice: 2
Stack Underflow!
Enter your choice: 5
Program terminated.
```

Overflow test (6 pushes into MAX = 5):

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Push
2. Pop
```

```
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 1
1 pushed into stack.
Enter your choice: 1
Enter value to push: 2
2 pushed into stack.
Enter your choice: 1
Enter value to push: 3
3 pushed into stack.
Enter your choice: 1
Enter value to push: 4
4 pushed into stack.
Enter your choice: 1
Enter value to push: 5
5 pushed into stack.
Enter your choice: 1
Enter value to push: 6
Stack Overflow!
Enter your choice: 5
Program terminated.
```

WRITE IN YOUR RECORD

Aim: To implement a stack using an array with push, pop, peek and display operations.

Algorithm: top = −1. *Push:* if top == MAX−1 → overflow; else top++, stackArr[top] = value. *Pop:* if top == −1 → underflow; else print stackArr[top], top−−. *Peek:* print stackArr[top]. *Display:* print from top down to 0.

Complexity: push, pop, peek O(1); display O(n); space O(MAX).

Result: Pushed 10, 20, 30; popped 30, 20, 10 (LIFO). Overflow on the 6th push and underflow on popping an empty stack were detected.

8.3 Viva questions

Question	Answer
What is LIFO?	Last In, First Out: the most recently added item is removed first.
Overflow / underflow?	Push onto a full stack (top == MAX−1) / pop from an empty stack (top == −1).
Why does top start at −1?	So the first push makes it 0, the first array index. −1 = empty.
Stack vs. queue?	Stack: one end, LIFO. Queue: insert at rear, delete at front, FIFO.
Applications?	Function calls and recursion, undo, expression evaluation (postfix), bracket matching, backtracking.
Does pop erase the value?	No, it only moves top down; the old value is overwritten by the next push.

CHECK 8.1

MAX = 3. Operations: push 4, push 7, pop, push 9, push 2, push 6. What does each print? What is top at the end?

Answers

Check 8.1: 4 pushed (top 0), 7 pushed (top 1), pop prints 7 (top 0), 9 pushed (top 1), 2 pushed (top 2), push 6 → **Stack Overflow!** (top == MAX−1 = 2). top ends at **2**, and the stack is 4 9 2 (bottom → top).